

## Problem 1: ConvNets Transfer Learning (5 points)

Goal: Build convolutional neural networks (ConvNets) based on ResNet101, InceptionV3, and EfficientNetB1, using transfer learning to classify images of fruits and vegetables into their respective classes. Analyze the performance of the constructed ConvNets of varying sizes in terms of classification accuracy as well as computational efforts required for inference.

Data: The Fruits-360 dataset contains 100×100 images for 251 different varieties of fruits and vegetables. Processing the images should be consistent with settings described in Homework 2.

```
In [1]: from google.colab import userdata

## IMPORT API KEY - UNCOMMENT APPLICABLE LINES WHEN RUNNING DIFFERENT KERNELS
# Import KAGGLE_API_KEY from .env
# KAGGLE_API_TOKEN = os.getenv("KAGGLE_API_KEY")
# Define directly - delete key before uploading to GitHub
# KAGGLE_API_TOKEN =
# When running in Colab - define in web using Colab Secrets
KAGGLE_API_TOKEN = userdata.get("KAGGLE_API_KEY")

# update any packages in Colab server
# %pip install --upgrade numpy pandas kagglehub torch torchvision IPython
```

```
In [2]: # Import necessary libraries
from pathlib import Path
from dotenv import load_dotenv
import os
import kagglehub
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import random_split, DataLoader
import torchvision
import torchvision.transforms as transforms
from IPython.display import display, Image
import time
import copy
from datetime import datetime
```

```
In [3]: # Import and Process Fruits-360 Dataset, same as HW2

# Import dataset from Kaggle
# Kagglehub reads KAGGLE_API_TOKEN from environment (set in cell above)
os.environ["KAGGLE_API_TOKEN"] = KAGGLE_API_TOKEN
# Running locally:
# path = kagglehub.dataset_download("moltean/fruits", output_dir="/content/fruits-3
```

```

# On Colab: use /content to avoid cache disk limits; Locally omit output_dir to use
path = kagglehub.dataset_download("moltean/fruits")

# Locate Training and Test subfolders (Fruits-360 dataset structure)
path = Path(path)
# Kaggle zip may nest Training/Test inside a subfolder (e.g. fruits-360/Training)
# Search recursively for Training directory
train_path = None
for p in path.rglob("Training"):
    if p.is_dir() and (p.parent / "Test").exists():
        train_path = p
        break
if train_path is not None:
    test_path = train_path.parent / "Test"
else:
    # Fallback: check direct children
    if (path / "Training").exists():
        train_path, test_path = path / "Training", path / "Test"
    else:
        raise FileNotFoundError(
            f"Could not find Training/Test folders. Dataset root: {path}\n"
            f"Contents: {[d.name for d in path.iterdir()] if path.exists() else 'path does not exist'}"
        )

# Load image datasets with placeholder transform (will add scaling/normalization in
print("Loading datasets")
train_dataset = torchvision.datasets.ImageFolder(str(train_path), transform=transform)
test_dataset = torchvision.datasets.ImageFolder(str(test_path), transform=transform)

print(f"Dataset downloaded to: {path}")
print(f"Training samples: {len(train_dataset)} | Classes: {len(train_dataset.classes)}")
print(f"Test samples: {len(test_dataset)}")

## Image Preprocessing

# 1. Image scaling (100x100 -> 299x299):
# Larger size for Inception
image_size = 299

# 2. Image normalization values for RGB to [-1, 1]
normalize_means = [0.5, 0.5, 0.5]
normalize_std = [0.5, 0.5, 0.5]

# 3. Data augmentation and normalization for training; only normalization for validation
train_transform = transforms.Compose([
    transforms.Resize((image_size, image_size)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=normalize_means, std=normalize_std)
])

test_transform = transforms.Compose([
    transforms.Resize((image_size, image_size)),
    transforms.ToTensor(),
    transforms.Normalize(mean=normalize_means, std=normalize_std)
])

```

```

# Replace raw datasets with transformed datasets
train_dataset = torchvision.datasets.ImageFolder(str(train_path), transform=train_t
test_dataset = torchvision.datasets.ImageFolder(str(test_path), transform=test_tran

# 4. Define training and validation split (85%/15% from train_dataset)
total_train = len(train_dataset)
val_size = int(0.15 * total_train)
train_size = total_train - val_size

SHUFFLE_SEED = 42
torch.manual_seed(SHUFFLE_SEED) # set random seed for reproducibility
train_subset, val_subset = random_split(train_dataset, [train_size, val_size], gene

# 5. Define dataloaders with batch size 64 and consistent shuffle with seed
# Smaller batch size due to larger image size
batch_size = 64

train_loader = DataLoader(train_subset, batch_size=batch_size, shuffle=True, worker
val_loader = DataLoader(val_subset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
print("Loading success")

```

Using Colab cache for faster access to the 'fruits' dataset.

Loading datasets

Dataset downloaded to: /kaggle/input/fruits

Training samples: 132448 | Classes: 253

Test samples: 44138

Loading success

```

In [4]: ## GENERAL HELPER FUNCTIONS

# BASIC TRAIN
def train(model, loader, criterion, optimizer, device):
    """
    Train the model for one epoch.

    Parameters:
        model (nn.Module): The neural network model.
        loader (DataLoader): DataLoader for training data.
        criterion: Loss function.
        optimizer: Optimization algorithm.
        device: Device to run the training on.

    Returns:
        epoch_loss (float): Average loss for the epoch.
        epoch_acc (float): Accuracy for the epoch.
    """
    model.train() # Set model to training mode
    running_loss = 0.0
    correct = 0
    total = 0

    for inputs, labels in loader:
        inputs, labels = inputs.to(device), labels.to(device)

```

```

optimizer.zero_grad() # Zero the parameter gradients

outputs = model(inputs) # Forward pass
loss = criterion(outputs, labels) # Compute Loss
loss.backward() # Backward pass
optimizer.step() # Optimize

running_loss += loss.item() * inputs.size(0) # Accumulate Loss

# Calculate accuracy
_, predicted = torch.max(outputs.data, 1)
total += labels.size(0)
correct += (predicted == labels).sum().item()

epoch_loss = running_loss / len(loader.dataset)
epoch_acc = correct / total
return epoch_loss, epoch_acc

# BASIC EVAL
def evaluate(model, loader, criterion, device):
    # Compute 3 accuracy percentages
    """
    Evaluate the model on validation or test data.

    Parameters:
        model (nn.Module): The neural network model.
        loader (DataLoader): DataLoader for validation/test data.
        criterion: Loss function.
        device: Device to run the evaluation on.

    Returns:
        epoch_loss (float): Average loss for the epoch.
        epoch_acc (float): Accuracy for the epoch.
    """
    model.eval() # Set model to evaluation mode
    running_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad(): # Disable gradient calculation
        for inputs, labels in loader:
            inputs, labels = inputs.to(device), labels.to(device)

            outputs = model(inputs) # Forward pass
            loss = criterion(outputs, labels) # Compute Loss

            running_loss += loss.item() * inputs.size(0) # Accumulate Loss

            # Calculate accuracy
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(loader.dataset)
    epoch_acc = correct / total
    return epoch_loss, epoch_acc

```

```

# PLOT LOSS CURVES
def plot_loss(curve, dataset, model_name, output_dir="Plot JPGs"):
    # Plot and save loss curve
    # dataset = [Training, Validation]
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    plt.figure()
    plt.plot(curve)
    plt.xlabel("Epoch")
    if dataset == 'Training':
        plt.ylabel("Training Loss")
    elif dataset == 'Validation':
        plt.ylabel("Validation Loss")

    plt.title(f"Categorical Cross Entropy Loss Curve for {dataset} Dataset, Model:
filename = f"{output_dir}/LossCurve_{dataset}_Dataset_Model_{model_name}.jpg"

    plt.savefig(filename)
    plt.close()

# MODEL TRAIN
def train_model(model, train_loader, test_loader, criterion, optimizer, device, num
"""
    Train the model and evaluate on test data after each epoch.

    Parameters:
        model (nn.Module): The neural network model.
        train_loader (DataLoader): DataLoader for training data.
        test_loader (DataLoader): DataLoader for test data.
        criterion: Loss function.
        optimizer: Optimization algorithm.
        device: Device to run the training on.
        num_epochs (int): Number of training epochs.

    Returns:
        Training and validation loss and accuracy.
    """
    training_loss_curve = []
    validation_loss_curve = []
    training_accuracy = []
    validation_accuracy = []

    for epoch in range(num_epochs):
        train_loss, train_acc = train(model, train_loader, criterion, optimizer, de
        val_loss, val_acc = evaluate(model, test_loader, criterion, device)

        training_loss_curve.append(train_loss)
        training_accuracy.append(train_acc)
        validation_loss_curve.append(val_loss)
        validation_accuracy.append(val_acc)

        timestamp = datetime.now().strftime("%H:%M:%S")
        print(f'[{timestamp}] Epoch {epoch+1}/{num_epochs} | '
              f'Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | '

```

```
f'Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}')
```

```
return training_loss_curve, validation_loss_curve, training_accuracy, validation
```

## Model Architecture

Background: The learned features in pre-trained models can be repurposed or transferred to another ConvNet designed for a specific task. Several complex pre-trained ConvNet models are made available by popular deep learning libraries. The pre-trained neural networks of varying sizes, Efficient Net B1, Inception V3, and ResNet 101 neural networks, could be used to classify images of fruits and vegetables. Weights corresponding to pre-training on ImageNet should be used as the initial weights for your pre-trained models.

Architecture: Because the above-mentioned ConvNets were previously trained to classify different numbers of image classes, the constructed models need to be customized such that they can classify 251 classes of fruits and vegetables. To achieve this, an additional Dense layer with 251 units and Softmax activation function can be added at the end of the pre-trained models. As the pre-trained ConvNets have sophisticated architectures and consist of several cyclic networks, sequential style modeling cannot be used here. Instead, the modified ConvNets for classifying fruits/vegetables can be built following an alternate modeling scheme termed as the functional model. In this assignment, you will need to:

1. Implement ResNet101, InceptionV3, and EfficientNetB1 models with a custom fully-connected layer (number\_of\_classes=251).
2. Incorporate pre-trained weights obtained from ImageNet for all three ConvNets. Proceed to train the constructed models while freezing the original weights of the convolutional blocks.
3. Freeze the weights of the original layers within the pre-trained models, and subsequently fine-tune the remaining layers. After retraining the modified models, elaborate on the roles that transfer learning and frozen layers play within transfer learning tasks.
4. Measure the computational time required for the prediction step over the testing dataset performed with all three trained ConvNets.

Training: All models should be trained with the ADAM optimizer to minimize the categorical cross-entropy. The new ConvNet models should be trained for 50 epochs (or 20ish if training is compute-intensive) using the data generators created for the fruits-360 dataset. Please note that some pre-trained models are large and might consume large computational resources for each step. It is recommended that the training process is performed on Google Colab / Google Cloud using a GPU or a TPU.

```
In [5]: ## 1. Set up ResNet101, InceptionV3, and EfficientNetB1 models with a custom fully-  
NUM_CLASSES = 253  
# ResNet101
```

```

ResNet101 = torchvision.models.resnet101(pretrained=True)
# Replace the final fully-connected layer to match number of classes
in_features_resnet = ResNet101.fc.in_features
ResNet101.fc = nn.Linear(in_features_resnet, NUM_CLASSES)

# InceptionV3
# Use pretrained weights; disable auxiliary logits for simplicity during training
InceptionV3 = torchvision.models.inception_v3(pretrained=True)
InceptionV3.aux_logits = False
in_features_inception = InceptionV3.fc.in_features
InceptionV3.fc = nn.Linear(in_features_inception, NUM_CLASSES)

# EfficientNetB1
EfficientNetB1 = torchvision.models.efficientnet_b1(pretrained=True)
# EfficientNetB1's classifier is a Sequential; replace the final Linear layer
in_features_effnet = EfficientNetB1.classifier[-1].in_features
EfficientNetB1.classifier[-1] = nn.Linear(in_features_effnet, NUM_CLASSES)

```

/usr/local/lib/python3.12/dist-packages/torchvision/models/\_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.

warnings.warn(

/usr/local/lib/python3.12/dist-packages/torchvision/models/\_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=ResNet101\_Weights.IMAGENET1K\_V1`. You can also use `weights=ResNet101\_Weights.DEFAULT` to get the most up-to-date weights.

warnings.warn(msg)

Downloading: "https://download.pytorch.org/models/resnet101-63fe2227.pth" to /root/.cache/torch/hub/checkpoints/resnet101-63fe2227.pth

100%|██████████| 171M/171M [00:00<00:00, 249MB/s]

/usr/local/lib/python3.12/dist-packages/torchvision/models/\_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=Inception\_V3\_Weights.IMAGENET1K\_V1`. You can also use `weights=Inception\_V3\_Weights.DEFAULT` to get the most up-to-date weights.

warnings.warn(msg)

Downloading: "https://download.pytorch.org/models/inception\_v3\_google-0cc3c7bd.pth" to /root/.cache/torch/hub/checkpoints/inception\_v3\_google-0cc3c7bd.pth

100%|██████████| 104M/104M [00:00<00:00, 239MB/s]

/usr/local/lib/python3.12/dist-packages/torchvision/models/\_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=EfficientNet\_B1\_Weights.IMAGENET1K\_V1`. You can also use `weights=EfficientNet\_B1\_Weights.DEFAULT` to get the most up-to-date weights.

warnings.warn(msg)

Downloading: "https://download.pytorch.org/models/efficientnet\_b1\_rwrightman-bac287d4.pth" to /root/.cache/torch/hub/checkpoints/efficientnet\_b1\_rwrightman-bac287d4.pth

100%|██████████| 30.1M/30.1M [00:00<00:00, 181MB/s]

```

In [6]: # First Experiment HELPER
def run_first_experiment(model, model_name, train_loader, val_loader, test_loader,
    # Run full experiment: instantiate model, train, evaluate accuracies, plot, ret
    # Train constructed models with pre-trained ImageNet weights and custom FC Layer

    # Detect Device

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Freeze all original layers
for param in model.parameters():
    param.requires_grad = False

# Unfreeze only the non-original layers (new classifier head)
# ResNet / Inception use `.fc`, EfficientNet uses `.classifier`.
if hasattr(model, "fc") and isinstance(model.fc, nn.Module):
    for param in model.fc.parameters():
        param.requires_grad = True
if hasattr(model, "classifier") and isinstance(model.classifier, nn.Module):
    for param in model.classifier.parameters():
        param.requires_grad = True

# Move model to device
model.to(device)

# Define Optimizer
# Adam, Categorical Cross Entropy
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()))

# Train
training_loss_curve, validation_loss_curve, training_accuracy, validation_accuracy = train(
    model, train_loader, val_loader, criterion, optimizer, device, max_epochs
)
print("Training complete")
# Evaluate final test
final_test_loss, test_acc = evaluate(model, test_loader, criterion, device)
print("Evaluation complete")
# Save plots
plot_loss(training_loss_curve, "Training", model_name, output_dir="Plot JPGs Ex")
plot_loss(validation_loss_curve, "Validation", model_name, output_dir="Plot JPGs Ex")
print("Plots created")
# return results
row = {
    'Model Name': model_name,
    'Epochs': max_epochs,
    'Training Accuracy (%)': training_accuracy[-1] * 100.0,
    'Validation Accuracy (%)': validation_accuracy[-1] * 100.0,
    'Test Accuracy (%)': test_acc * 100.0,
    'Final Test Loss': final_test_loss
}
timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Experiment complete for {model_name}")
return row

```

In [7]: `## 2. Train the 3x models with frozen original layers and pre-trained ImageNet weights`  
`# Report: Training accuracy, validation accuracy, training Loss, validation Loss`

```
MAX_EPOCHS = 5
```

```
first_experiment_df = pd.DataFrame(columns=[
    'Model Name', 'Epochs', 'Training Accuracy (%)', 'Validation Accuracy (%)', 'Test Accuracy (%)', 'Final Test Loss']

```



```

))

timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Run Start")
first_experiment_df.loc[len(first_experiment_df)] = run_first_experiment(ResNet101,

```

Using device: cuda

[15:30:40] Epoch 1/5 | Train Loss: 0.4579 | Train Acc: 0.9299 | Val Loss: 0.0474 | Val Acc: 0.9932

[16:04:31] Epoch 2/5 | Train Loss: 0.0419 | Train Acc: 0.9925 | Val Loss: 0.0223 | Val Acc: 0.9955

[16:38:19] Epoch 3/5 | Train Loss: 0.0254 | Train Acc: 0.9943 | Val Loss: 0.0428 | Val Acc: 0.9886

[17:11:55] Epoch 4/5 | Train Loss: 0.0252 | Train Acc: 0.9929 | Val Loss: 0.0122 | Val Acc: 0.9972

[17:45:41] Epoch 5/5 | Train Loss: 0.0177 | Train Acc: 0.9949 | Val Loss: 0.0158 | Val Acc: 0.9951

Training complete

Evaluation complete

Plots created

[18:00:13] Experiment complete for ResNet101

```

In [8]: timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Run Start")
first_experiment_df.loc[len(first_experiment_df)] = run_first_experiment(InceptionV

```

Using device: cuda

[18:17:36] Epoch 1/5 | Train Loss: 0.8064 | Train Acc: 0.8663 | Val Loss: 0.1243 | Val Acc: 0.9794

[18:34:56] Epoch 2/5 | Train Loss: 0.1621 | Train Acc: 0.9632 | Val Loss: 0.0558 | Val Acc: 0.9902

[18:52:01] Epoch 3/5 | Train Loss: 0.1184 | Train Acc: 0.9687 | Val Loss: 0.0379 | Val Acc: 0.9928

[19:08:58] Epoch 4/5 | Train Loss: 0.1005 | Train Acc: 0.9704 | Val Loss: 0.0288 | Val Acc: 0.9930

[19:25:57] Epoch 5/5 | Train Loss: 0.0909 | Train Acc: 0.9724 | Val Loss: 0.0276 | Val Acc: 0.9925

Training complete

Evaluation complete

Plots created

[19:31:21] Experiment complete for InceptionV3

```

In [9]: timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Run Start")
first_experiment_df.loc[len(first_experiment_df)] = run_first_experiment(EfficientN

```

```

Using device: cuda
[19:47:20] Epoch 1/5 | Train Loss: 0.4341 | Train Acc: 0.9483 | Val Loss: 0.0378 | Val Acc: 0.9973
[20:03:21] Epoch 2/5 | Train Loss: 0.0419 | Train Acc: 0.9947 | Val Loss: 0.0174 | Val Acc: 0.9986
[20:18:43] Epoch 3/5 | Train Loss: 0.0236 | Train Acc: 0.9963 | Val Loss: 0.0098 | Val Acc: 0.9989
[20:34:30] Epoch 4/5 | Train Loss: 0.0159 | Train Acc: 0.9972 | Val Loss: 0.0056 | Val Acc: 0.9993
[20:50:11] Epoch 5/5 | Train Loss: 0.0121 | Train Acc: 0.9975 | Val Loss: 0.0042 | Val Acc: 0.9996
Training complete
Evaluation complete
Plots created
[20:54:57] Experiment complete for EfficientNetB1

```

```

In [ ]: ## DOWNLOAD PLOT FOLDERS TO LOCAL IN CASE OF DISCONNECT

import shutil
from google.colab import files
import os

# Define the names for the downloaded zip files
zip_ex1 = 'Plot_JPGs_Ex1'

# Check if directories exist on the server and zip them
if os.path.exists("/content/Plot JPGs Ex1"):
    shutil.make_archive(zip_ex1, 'zip', "/content/Plot JPGs Ex1")
    print(f"Created {zip_ex1}.zip")
    files.download(f"{zip_ex1}.zip")
else:
    print("Folder '/content/Plot JPGs Ex1' not found. Make sure the first experimen

```

```

In [10]: # Second Experiment HELPER (fine-tuning non-original layers)
def run_second_experiment(model, model_name, train_loader, val_loader, test_loader,
# Detect Device
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device} for fine-tuning {model_name}")

    # Freeze all original layers
    for param in model.parameters():
        param.requires_grad = False

    # Unfreeze the last block of each model + FC Layer
    # ResNet101: unfreeze model.layer4
    # InceptionV3: unfreeze model.Mixed_7a, Mixed_7b, Mixed_7c
    # EfficientNetB1: unfreeze last few blocks in model.features[-3:]
    # ResNet / Inception use .fc, EfficientNet uses .classifier

    # --- ResNet101: unfreeze last residual block (layer4) and classifier ---
    if hasattr(model, "layer4") and isinstance(model.layer4, nn.Module):
        for param in model.layer4.parameters():
            param.requires_grad = True
    if hasattr(model, "fc") and isinstance(model.fc, nn.Module):
        for param in model.fc.parameters():
            param.requires_grad = True

```

```

# --- InceptionV3: unfreeze final inception blocks and classifier ---
for attr in ["Mixed_7a", "Mixed_7b", "Mixed_7c"]:
    if hasattr(model, attr):
        block = getattr(model, attr)
        if isinstance(block, nn.Module):
            for param in block.parameters():
                param.requires_grad = True
if hasattr(model, "fc") and isinstance(model.fc, nn.Module):
    for param in model.fc.parameters():
        param.requires_grad = True

# --- EfficientNetB1: unfreeze last few feature blocks and classifier ---
if hasattr(model, "features") and isinstance(model.features, nn.Module):
    # Unfreeze parameters belonging to the last 3 feature blocks
    params = list(model.features.parameters())
    for param in params[-3:]:
        param.requires_grad = True
if hasattr(model, "classifier") and isinstance(model.classifier, nn.Module):
    for param in model.classifier.parameters():
        param.requires_grad = True

# Move model to device
model.to(device)

# Use a smaller learning rate for fine-tuning
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()), 1e-5)

# Train
training_loss_curve, validation_loss_curve, training_accuracy, validation_accuracy = train(
    model, train_loader, val_loader, criterion, optimizer, device, max_epochs
)
print("Training complete")
# Evaluate final test
final_test_loss, test_acc = evaluate(model, test_loader, criterion, device)
print("Evaluation complete")
# Save plots
plot_loss(training_loss_curve, "Training", model_name, output_dir="Plot JPGs Experiment")
plot_loss(validation_loss_curve, "Validation", model_name, output_dir="Plot JPGs Experiment")
print("Plots made")
# return results
row = {
    'Model Name': model_name,
    'Epochs': max_epochs,
    'Training Accuracy (%)': training_accuracy[-1] * 100.0,
    'Validation Accuracy (%)': validation_accuracy[-1] * 100.0,
    'Test Accuracy (%)': test_acc * 100.0,
    'Final Test Loss': final_test_loss
}
timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Experiment complete for {model_name}")
return row

```

In [13]: *## 3. Freeze weights of original layers for fine-tune training the remaining layers*  
*# Include logic to measure inference time in seconds*

```

MAX_EPOCHS = 5

# Create copies of trained models for fine-tuning in Experiment 2
ResNet101_fine = copy.deepcopy(ResNet101)
InceptionV3_fine = copy.deepcopy(InceptionV3)
EfficientNetB1_fine = copy.deepcopy(EfficientNetB1)
# Unfreeze the last layer within the experiment function

second_experiment_df = pd.DataFrame(columns=[
    'Model Name', 'Epochs', 'Training Accuracy (%)', 'Validation Accuracy (%)', 'Te
])

timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Run Start")
# Use the fine-tuning copies created after Experiment 1
second_experiment_df.loc[len(second_experiment_df)] = run_second_experiment(ResNet1

```

```

[22:45:05] Run Start
Using device: cuda for fine-tuning ResNet101_fine
[23:21:53] Epoch 1/5 | Train Loss: 0.0170 | Train Acc: 0.9959 | Val Loss: 0.0104 | V
al Acc: 0.9970
[23:59:49] Epoch 2/5 | Train Loss: 0.0073 | Train Acc: 0.9982 | Val Loss: 0.0235 | V
al Acc: 0.9947
[00:37:53] Epoch 3/5 | Train Loss: 0.0052 | Train Acc: 0.9991 | Val Loss: 0.0013 | V
al Acc: 0.9999
[01:16:04] Epoch 4/5 | Train Loss: 0.0044 | Train Acc: 0.9989 | Val Loss: 0.0077 | V
al Acc: 0.9983
[01:54:12] Epoch 5/5 | Train Loss: 0.0032 | Train Acc: 0.9994 | Val Loss: 0.0275 | V
al Acc: 0.9956
Training complete
Evaluation complete
Plots made
[02:04:41] Experiment complete for ResNet101_fine

```

```

In [14]: timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Run Start")
second_experiment_df.loc[len(second_experiment_df)] = run_second_experiment(Incepti

```

```

[02:04:41] Run Start
Using device: cuda for fine-tuning InceptionV3_fine
[02:24:58] Epoch 1/5 | Train Loss: 0.0366 | Train Acc: 0.9887 | Val Loss: 0.0053 | V
al Acc: 0.9985
[02:45:11] Epoch 2/5 | Train Loss: 0.0125 | Train Acc: 0.9964 | Val Loss: 0.0021 | V
al Acc: 0.9995
[03:05:27] Epoch 3/5 | Train Loss: 0.0101 | Train Acc: 0.9969 | Val Loss: 0.0036 | V
al Acc: 0.9991
[03:25:41] Epoch 4/5 | Train Loss: 0.0079 | Train Acc: 0.9979 | Val Loss: 0.0013 | V
al Acc: 0.9998
[03:45:18] Epoch 5/5 | Train Loss: 0.0058 | Train Acc: 0.9984 | Val Loss: 0.0012 | V
al Acc: 0.9998
Training complete
Evaluation complete
Plots made
[03:50:30] Experiment complete for InceptionV3_fine

```

```
In [15]: timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Run Start")
second_experiment_df.loc[len(second_experiment_df)] = run_second_experiment(EfficientNetB1_fine)
```

```
[03:50:30] Run Start
Using device: cuda for fine-tuning EfficientNetB1_fine
[04:06:22] Epoch 1/5 | Train Loss: 0.0079 | Train Acc: 0.9984 | Val Loss: 0.0026 | Val Acc: 0.9996
[04:20:53] Epoch 2/5 | Train Loss: 0.0055 | Train Acc: 0.9990 | Val Loss: 0.0017 | Val Acc: 0.9998
[04:35:09] Epoch 3/5 | Train Loss: 0.0048 | Train Acc: 0.9990 | Val Loss: 0.0013 | Val Acc: 0.9999
[04:50:46] Epoch 4/5 | Train Loss: 0.0036 | Train Acc: 0.9993 | Val Loss: 0.0011 | Val Acc: 0.9999
[05:06:36] Epoch 5/5 | Train Loss: 0.0032 | Train Acc: 0.9994 | Val Loss: 0.0009 | Val Acc: 0.9999
Training complete
Evaluation complete
Plots made
[05:11:21] Experiment complete for EfficientNetB1_fine
```

```
In [16]: ## DOWNLOAD PLOT FOLDERS TO LOCAL IN CASE OF DISCONNECT

import shutil
from google.colab import files
import os

# Define the names for the downloaded zip files
zip_ex2 = 'Plot_JPGs_Ex2'

# Check if directories exist on the server and zip them

if os.path.exists("/content/Plot JPGs Ex2"):
    shutil.make_archive(zip_ex2, 'zip', "/content/Plot JPGs Ex2")
    print(f"Created {zip_ex2}.zip")
    files.download(f"{zip_ex2}.zip")
else:
    print("Folder '/content/Plot JPGs Ex2' not found. Make sure the second experiment is completed.")
```

Created Plot\_JPGs\_Ex2.zip

```
In [17]: # Helper function to measure inference time over test dataset

def measure_inference_time(model, test_loader, device):
    model.eval()
    model.to(device)

    correct = 0
    total = 0

    if device.type == "cuda":
        torch.cuda.synchronize()

    start = time.perf_counter()

    with torch.no_grad():
```

```

    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    if device.type == "cuda":
        torch.cuda.synchronize()

    end = time.perf_counter()
    inference_time = end - start
    test_accuracy = correct / total

    return inference_time, test_accuracy

```

```

In [18]: ## 4. Prediction and Measure Computational Time
# Report: Testing accuracy, prediction time, model size

# Models to run
models_to_run = [ResNet101_fine, InceptionV3_fine, EfficientNetB1_fine]

prediction_results_df = pd.DataFrame(columns=[
    'Model', 'Model Size (num. params)', 'Inference Time (sec)', 'Testing Accuracy
'])

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
timestamp = datetime.now().strftime("%H:%M:%S")
print(f"[{timestamp}] Run Start")

for model in models_to_run:
    inference_time, test_acc = measure_inference_time(model, test_loader, device)
    model_size = sum(p.numel() for p in model.parameters())

    prediction_results_df.loc[len(prediction_results_df)] = {
        "Model": model.__class__.__name__,
        "Model Size (num. params)": model_size,
        "Inference Time (sec)": inference_time,
        "Testing Accuracy (%)": test_acc * 100.0,
    }

```

[05:11:21] Run Start

Deliverables:

1. Please report the accuracy for both the training and testing datasets obtained after training the modified ConvNets. Also, plot the loss curves for both training and validation datasets. Discuss the roles that transfer learning and frozen layers play within transfer learning tasks.
2. Report the computational time required to predict classification accuracy over the testing dataset with the trained ConvNet models along with their model sizes. Please make sure to measure ONLY the times required for the prediction step; you may use Python's inbuilt time functions for the same. Discuss the trade-offs between

computational resource utilization with the accuracy levels and model sizes for the classification task.  alt text

3. Please make sure to submit your working code files along with the final results and the plots.

```
In [19]: # Deliverable 1: Models with ImageNet weights and custom FC Layer
# Dataframe: first_experiment_df

# Report accuracies
# Display as a clean HTML table (use display() in Jupyter instead of print())
try:
    display(first_experiment_df.style.hide(axis=0).set_properties(
        **{'text-align': 'center', 'border': '1px solid #ddd'}
    ).set_table_styles([{'selector': 'th', 'props': [('text-align', 'center')], ('fo
except NameError as e:
    print(f"Error: {e}. Run the model training cells first.")

# Print loss curves
# Display in grid 3x2

# Define subfolder and filenames
# If local:
# plot_folder = "HW_03_Transfer/plots"
# If running in Colab online:
plot_folder = "/content/Plot JPGs Ex1"

# Debug: List files to ensure we have the right names
if os.path.exists(plot_folder):
    print(f"Files in {plot_folder}: {os.listdir(plot_folder)}")
else:
    print(f"Folder {plot_folder} not found!")

# Rows: Training, Validation
# Cols: Models

models = ["ResNet101", "InceptionV3", "EfficientNetB1"]
datasets = ["Training", "Validation"]

# Increased figure height to 12 to allow more space
fig, axs = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle("Loss Curves (Top: Training, Bottom: Validation)", fontsize=16)

for row_idx, dataset in enumerate(datasets):
    for col_idx, model in enumerate(models):
        # Filename format from plot_loss: LossCurve_{dataset}_Dataset_Model_{model}
        fname = f"LossCurve_{dataset}_Dataset_Model_{model}.jpg"
        img_path = os.path.join(plot_folder, fname)
        ax = axs[row_idx, col_idx]

        if os.path.exists(img_path):
            img = mpimg.imread(img_path)
            ax.imshow(img)
            ax.set_title(f"{dataset} Loss (Model={model})", fontsize=12)
```

```

        ax.axis('off')
    else:
        ax.text(0.5, 0.5, f"File not found:\n{fname}", ha='center', va='center')
        ax.axis('off')

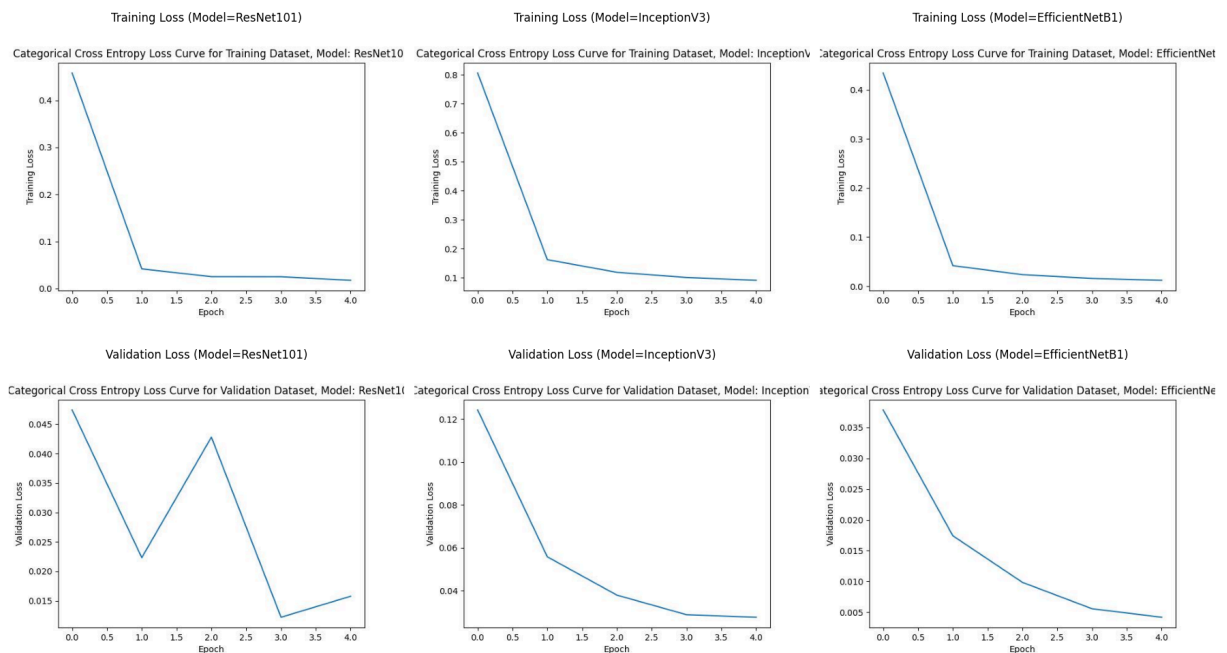
# Added h_pad=3.0 to increase vertical spacing between subplots
plt.tight_layout(rect=[0, 0.03, 1, 0.95], h_pad=3.0)
plt.show()

```

| Model Name     | Epochs | Training Accuracy (%) | Validation Accuracy (%) | Test Accuracy (%) | Final Test Loss |
|----------------|--------|-----------------------|-------------------------|-------------------|-----------------|
| ResNet101      | 5      | 99.491033             | 99.511753               | 97.387738         | 0.268565        |
| InceptionV3    | 5      | 97.241986             | 99.250013               | 96.393131         | 0.296943        |
| EfficientNetB1 | 5      | 99.747737             | 99.964766               | 98.459377         | 0.223698        |

Files in /content/Plot JPGs Ex1: ['LossCurve\_Training\_Dataset\_Model\_ResNet101.jpg', 'LossCurve\_Validation\_Dataset\_Model\_ResNet101.jpg', 'LossCurve\_Validation\_Dataset\_Model\_InceptionV3.jpg', 'LossCurve\_Validation\_Dataset\_Model\_EfficientNetB1.jpg', 'LossCurve\_Training\_Dataset\_Model\_InceptionV3.jpg', 'LossCurve\_Training\_Dataset\_Model\_EfficientNetB1.jpg']

Loss Curves (Top: Training, Bottom: Validation)



```

In [20]: # Deliverable 1.5: Fine tuned models
# Dataframe: second_experiment_df

# Report accuracies
# Display as a clean HTML table (use display() in Jupyter instead of print())
try:
    display(second_experiment_df.style.hide(axis=0).set_properties(
        **{'text-align': 'center', 'border': '1px solid #ddd'}
    ).set_table_styles([{'selector': 'th', 'props': [('text-align', 'center')], ('fo
except NameError as e:
    print(f"Error: {e}. Run the model training cells first.")

```



```

# Print loss curves
# Display in grid 3x2

# Define subfolder and filenames
# If Local:
# plot_folder = "HW_03_Transfer/plots"
# If running in Colab online:
plot_folder = "/content/Plot JPGs Ex2"

# Debug: List files to ensure we have the right names
if os.path.exists(plot_folder):
    print(f"Files in {plot_folder}: {os.listdir(plot_folder)}")
else:
    print(f"Folder {plot_folder} not found!")

# Rows: Training, Validation
# Cols: Models

models = ["ResNet101_fine", "InceptionV3_fine", "EfficientNetB1_fine"]
datasets = ["Training", "Validation"]

# Increased figure height to 12 to allow more space
fig, axs = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle("Loss Curves (Top: Training, Bottom: Validation)", fontsize=16)

for row_idx, dataset in enumerate(datasets):
    for col_idx, model in enumerate(models):
        # Filename format from plot_loss: LossCurve_{dataset}_Dataset_Model_{model}
        fname = f"LossCurve_{dataset}_Dataset_Model_{model}.jpg"
        img_path = os.path.join(plot_folder, fname)
        ax = axs[row_idx, col_idx]

        if os.path.exists(img_path):
            img = mpimg.imread(img_path)
            ax.imshow(img)
            ax.set_title(f"{dataset} Loss (Model={model})", fontsize=12)
            ax.axis('off')
        else:
            ax.text(0.5, 0.5, f"File not found:\n{fname}", ha='center', va='center')
            ax.axis('off')

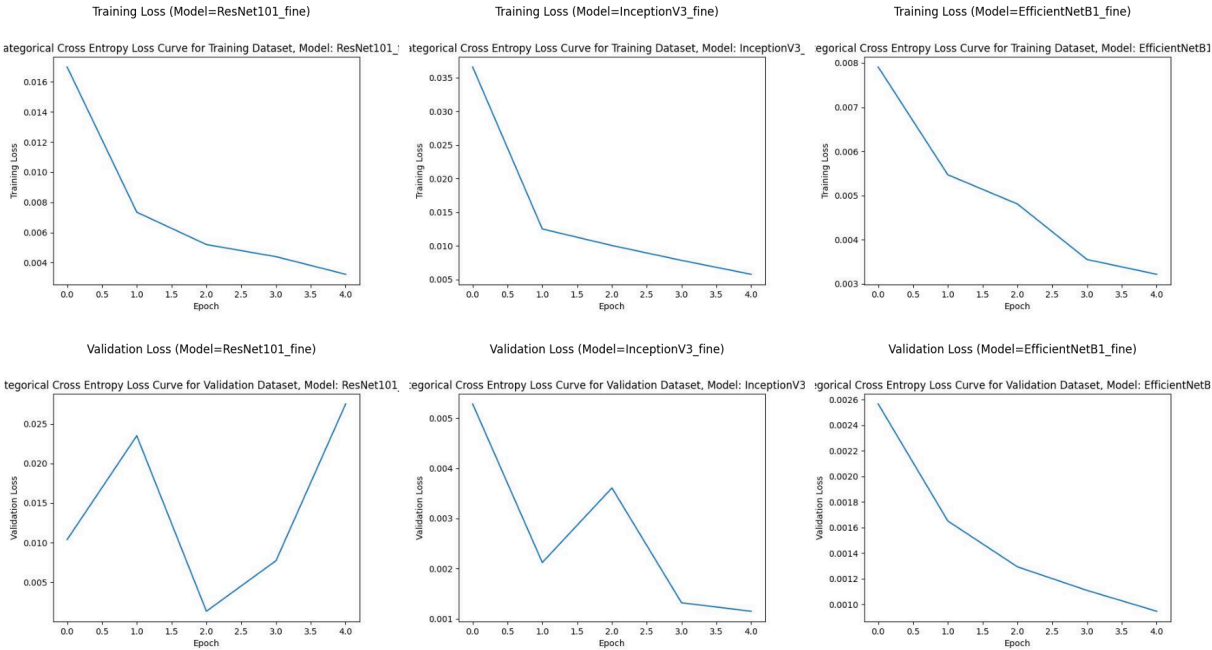
# Added h_pad=3.0 to increase vertical spacing between subplots
plt.tight_layout(rect=[0, 0.03, 1, 0.95], h_pad=3.0)
plt.show()

```

| Model Name          | Epochs | Training Accuracy (%) | Validation Accuracy (%) | Test Accuracy (%) | Final Test Loss |
|---------------------|--------|-----------------------|-------------------------|-------------------|-----------------|
| ResNet101_fine      | 5      | 99.936934             | 99.562088               | 98.017581         | 0.284283        |
| InceptionV3_fine    | 5      | 99.836562             | 99.984900               | 98.411799         | 0.357166        |
| EfficientNetB1_fine | 5      | 99.939599             | 99.994967               | 98.502424         | 0.246408        |

Files in /content/Plot JPGs Ex2: ['LossCurve\_Training\_Dataset\_Model\_EfficientNetB1\_fine.jpg', 'LossCurve\_Validation\_Dataset\_Model\_InceptionV3\_fine.jpg', 'LossCurve\_Training\_Dataset\_Model\_ResNet101\_fine.jpg', 'LossCurve\_Validation\_Dataset\_Model\_EfficientNetB1\_fine.jpg', 'LossCurve\_Validation\_Dataset\_Model\_ResNet101\_fine.jpg', 'LossCurve\_Training\_Dataset\_Model\_InceptionV3\_fine.jpg']

Loss Curves (Top: Training, Bottom: Validation)



```
In [21]: # Deliverable 2: Prediction time table
# Dataframe: prediction_results_df

# Display as a clean HTML table (use display() in Jupyter instead of print())
try:
    display(prediction_results_df.style.hide(axis=0).set_properties(
        **{'text-align': 'center', 'border': '1px solid #ddd'}
    ).set_table_styles([{'selector': 'th', 'props': [('text-align', 'center')], ('fo
except NameError as e:
    print(f"Error: {e}. Run the model evaluation cells first.")
```

| Model        | Model Size (num. params) | Inference Time (sec) | Testing Accuracy (%) |
|--------------|--------------------------|----------------------|----------------------|
| ResNet       | 43018557                 | 623.151427           | 98.017581            |
| Inception3   | 25630661                 | 320.585165           | 98.411799            |
| EfficientNet | 6837277                  | 283.518054           | 98.502424            |

Discussion Section

Role of Transfer Learning and Frozen Layers:

Transfer learning allows each model to leverage features learned on the large-scale ImageNet dataset and apply them to the Fruits-360 classification task, which has far fewer training samples per class. By initializing with the ImageNet pre-trained weights, the model

backbone already encodes edges, textures, and shapes that generalize well to new image domains, reducing the amount of training data and epochs needed. Even with training only 5 epochs/model due to computational constraints, every model demonstrated measurable training improvement.

Freezing the model and only training either the final fully connected classification layer or the FC layer + the last feature block serves to ensure that the model does not overwrite itself to overspecialize to Fruits-360. Experiment 1 (only training the FC layer) initializes the FC layer to understand the outputs of each model. All 3 models achieved 96%+ test accuracy with only this trained FC layer. This fine-tuning improved test accuracy for every model: ResNet101 rose from 97.39% to 98.02% (+0.63 pp), InceptionV3 from 96.39% to 98.41% (+2.02 pp), and EfficientNetB1 from 98.46% to 98.50% (+0.04 pp). InceptionV3 benefited most from fine-tuning, suggesting its frozen features were less well-suited to this domain out of the box, while EfficientNetB1's features already transferred nearly perfectly.

### **Trade-offs: Computational Resources vs. Accuracy vs. Model Size:**

EfficientNet wins across parameter count, inference time, and test accuracy. It achieved the highest test accuracy and fastest inference time while being by far the smallest model. EfficientNet is roughly 6.3x smaller than ResNet101 and 3.7x smaller than InceptionV3. InceptionV3 falls in the middle. While substantially more compact than ResNet101, it runs inference in roughly half the time and achieves a higher test accuracy. ResNet101 occupies the least favorable position, with the most parameters, the slowest inference (over 2x slower than EfficientNetB1), and the lowest test accuracy.

Training time follows the same trend: epoch training took approximately 34 min/epoch for ResNet101, 17 min/epoch for InceptionV3, and 16 min/epoch for EfficientNetB1 during Experiment 1. Fine-tuning in Experiment 2 increased training times, especially for ResNet101 (~38 min/epoch), while EfficientNetB1 remained the most affordable at ~16 min/epoch.

### **Key Takeaways**

1. Transfer learning is highly effective for this dataset: all models exceeded 96% test accuracy with only 5 epochs of classifier-head training, demonstrating that ImageNet features transfer well to fruit/vegetable classification.
2. Fine-tuning provides diminishing but consistent gains. Unfreezing later layers improved all models, with the largest gain for InceptionV3 (+2.02 pp) and the smallest for EfficientNetB1 (+0.04 pp).
3. Model efficiency played a large role in training and inference time. EfficientNetB1 demonstrated a clear improvement with no trade-off in every metric, showing that a well-designed smaller architecture can outperform much larger models in accuracy, inference speed, and memory footprint simultaneously.